

Recovery-Enhanced Imitation Learning for Robust Multi-Task Robot Manipulation

Junkai Zhang

University of Wisconsin–Madison jzhang3257@wsic.edu

Abstract—Action-chunking policies execute smooth manipulation sequences but may continue an unsuitable chunk after the state departs from the demonstrations. We present Recovery-Enhanced Imitation Learning (REIM), which combines a task-conditioned Action Chunking with Transformers policy, a causal LSTM risk monitor, a supervised recovery policy, and hysteretic control arbitration. The monitor predicts near-term behavioral-deviation events, and recovery supervision comes from successful expert continuations at online trigger states. We evaluate separately trained models on the known-task, goal-observable Meta-World MT10 and MT50 suites using 50 episodes per task. At the strongest perturbation, three MT10 detector/recovery seeds average $60.8\% \pm 4.9\%$ task-macro success versus $40.8\% \pm 3.6\%$ for the seed-matched original heuristic gate; four MT50 seeds average $48.75\% \pm 1.05\%$ versus $35.10\% \pm 0.94\%$. Clean reference-model success is 97.6% and 92.24%. A canonical reference-model occupancy-matched control shows that the trigger-specific effect is task-set dependent: the MT10 difference is -0.2 points $[-3.6, 3.0]$, whereas MT50 retains +11.32 points $[9.44, 13.16]$.

Index Terms—robot learning, imitation learning, recovery policy, runtime monitoring, multi-task manipulation

I. Introduction

Imitation learning can produce effective manipulation controllers without an explicit dynamics model, yet closed-loop deployment remains sensitive to covariate shift [1]–[3]. Modern implicit, sequence, and diffusion policies better represent multimodal robot behavior [4]–[7], but action generation alone does not decide when a nominal policy should relinquish control. In long-horizon manipulation, continuing a locally inappropriate chunk can convert a recoverable deviation into failure.

Prior recovery systems use expert intervention, hand-designed triggers, external safety signals, or reinforcement-learning objectives [8]–[10]. A recovery policy may be competent while its trigger is late; an aggressive trigger may instead interrupt nominal behavior. The central problem is therefore how to arbitrate between nominal and recovery control using causal information available at runtime.

We introduce Recovery-Enhanced Imitation Learning (REIM), a supervised recovery architecture for known-task multi-task manipulation. As illustrated in Fig. 1, a task-conditioned ACT policy supplies nominal action chunks, an LSTM consumes the most recent state history and estimates near-term behavioral-deviation risk, and a task-conditioned multilayer perceptron supplies recovery actions. A hysteretic state machine turns the continuous

risk estimate into stable control decisions. Separate entry and release conditions prevent rapid switching, a minimum dwell time gives recovery an opportunity to act, and a cooldown period prevents immediate retriggering.

The risk target predicts whether a task-calibrated disagreement event will occur in the immediate future rather than predicting terminal success. Hysteresis converts this ranking signal into stable switches, while the recovery policy is trained on states where intervention occurs. We evaluate known-task models on MT10 and MT50 under the same disturbance and evaluation protocol. The suites use separately trained models, so their results test consistency across task-set breadth rather than scaling of one pooled policy.

This work makes three contributions:

- a causal, task-conditioned risk monitor with validation-only operating-point selection and explicit hysteretic arbitration;
- a trigger-aligned supervised recovery dataset constructed from successful expert continuations at policy-induced states; and
- a paired evaluation across MT10 and MT50 that reports clean performance, a common perturbation sweep, rescued and harmed episodes, per-task heterogeneity, and an occupancy-matched trigger control.

II. Related Work

A. Imitation Learning and Action Chunking

Dataset aggregation and noise injection address covariate shift by collecting supervision near learner-induced states [1], [2], while offline manipulation studies emphasize demonstration quality and validation protocol [3]. Implicit BC, Behavior Transformers, and diffusion policies model multimodal actions [4]–[6]; ACT predicts and temporally ensembles action chunks [7], and MimicPlay uses hierarchical plans for long-horizon control [11]. REIM complements these policy models with trigger-aligned recovery rather than retraining the nominal policy.

B. Multi-Task Learning, Monitoring, and Recovery

PerAct and VIMA learn task-conditioned manipulation policies from structured or multimodal prompts, while MT-Opt scales shared robot learning through multi-task data [12]–[14]. Runtime monitors detect inconsistent policy behavior [15]; recovery has been learned through safe RL, residual control, meta-reasoning, and local policy

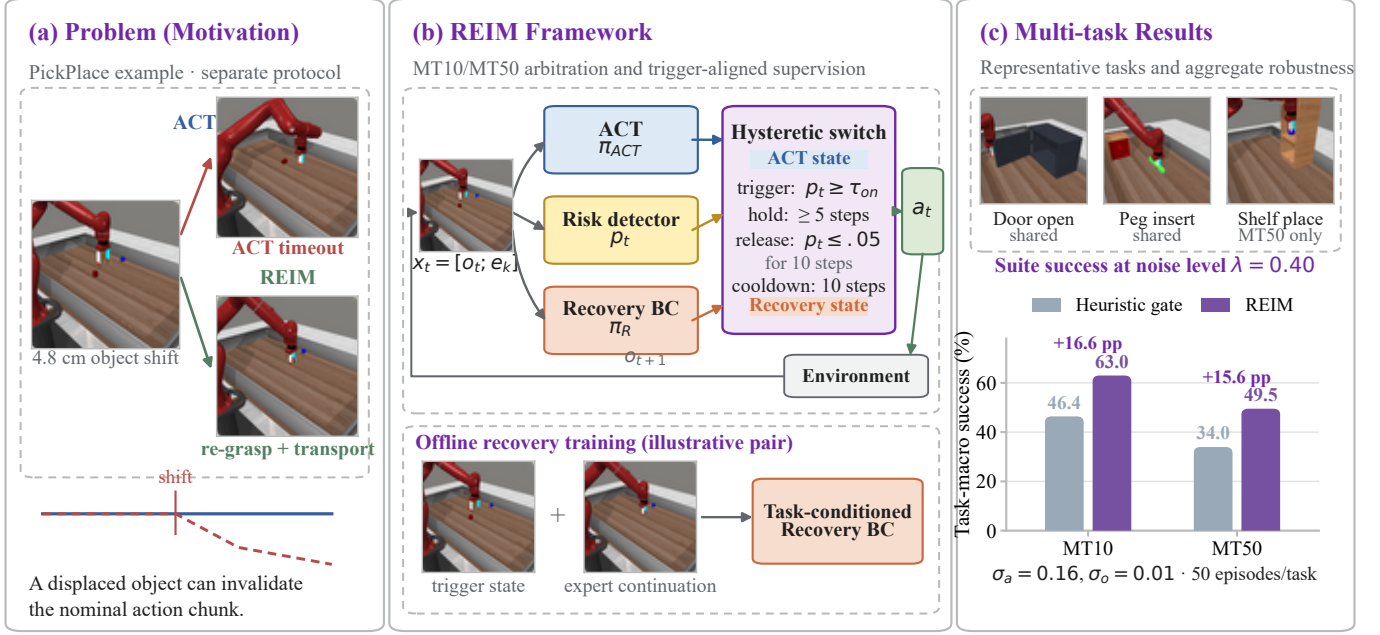


Fig. 1. REIM overview. (a) A separate PickPlace example motivates recovery after ACT drift. (b) Task-conditioned nominal control, causal risk monitoring, and hysteretic arbitration. (c) Representative MT tasks and reference-model results at $\lambda = 0.40$; MT10 and MT50 models are trained separately.

chaining [9], [10], [16], [17]. REIM instead trains both monitor and recovery policy by supervision and compares learned arbitration with reward-stagnation arbitration while holding the recovery policy fixed.

III. Method

A. Problem Formulation

Consider a manipulation suite with K known tasks. At step t , the environment provides a 39-dimensional state observation o_t and task identity k . We concatenate the state with a one-hot task code,

$$x_t = [o_t; e_k] \in \mathbb{R}^{39+K}, \quad (1)$$

so MT10 and MT50 use 49- and 89-dimensional policy inputs. An episode succeeds when the environment’s official success signal is observed within 500 steps. REIM combines a nominal policy π_N , risk monitor D , recovery policy π_R , and arbitration state $z_t \in \{N, R\}$.

B. Nominal Multi-Task ACT

The nominal controller is a state-based ACT policy [7]. Given x_t , it predicts a chunk of 20 four-dimensional actions. Training minimizes an ℓ_1 reconstruction loss plus the ACT latent KL term. Overlapping predictions are combined using exponential temporal ensembling. We train distinct ACT models for MT10 and MT50 because the task vocabularies and input dimensions differ. Both use three encoder layers, four decoder layers, eight attention heads, and KL weight 10; hidden/latent dimensions are 256/32 for MT10 and 384/48 for MT50.

C. Causal Near-Term Risk Monitor

The monitor receives only the latest $L = 10$ conditioned states. A one-layer LSTM [18] maps $x_{t-L+1:t}$ to a hidden representation, followed by a two-layer classifier and sigmoid output

$$p_t = \sigma(D(x_{t-L+1:t})). \quad (2)$$

Because no future observation is used at inference, the monitor is causal.

Risk labels are derived from action disagreement rather than terminal success alone. For each task, the 90th percentile of ℓ_1 disagreement between ACT and the scripted expert is fitted on the failure-training bank. A step is a risk event when its disagreement reaches that task-specific threshold. Failed rollouts also mark their final 25 steps as events. The binary target at time t is positive if an event occurs within the next 10 steps. Task thresholds fitted on the training bank are frozen for validation and final evaluation.

The detector is optimized with weighted binary cross entropy. Among validation thresholds with task-macro precision of at least 0.60, we select the highest task-macro F1, yielding $\tau_{on} = 0.65$ for MT10 and 0.64 for MT50. These metrics characterize event detection rather than task success.

D. Trigger-Aligned Recovery Policy

Recovery data are collected by running the nominal ACT policy and monitoring risk online. A deliberately broad collection threshold of 0.20 captures candidate

recovery starts. Once triggered, a scripted task expert controls the continuation. Only successful continuations are retained, yielding 50 trigger-aligned trajectories per task. This creates recovery supervision at states encountered by the learned nominal controller rather than only at states from clean demonstrations.

The recovery controller is a deterministic task-conditioned MLP with hidden dimensions 512–512–256 and Tanh activations. It uses Smooth- ℓ_1 action loss, state-noise augmentation of standard deviation 0.005, and task-balanced sampling. MT10 contains 500 recovery trajectories (30,807 training and 7,973 validation steps); MT50 contains 2,500 trajectories (159,581 training and 39,746 validation steps). A separate recovery model is trained for each suite.

All supervised models use AdamW and retain the lowest validation-loss checkpoint. ACT and MLP BC use batch size 512 with learning rates 10^{-4} and 3×10^{-4} ; ACT is capped at 120/100 epochs for MT10/MT50 and MLP BC at 120. The detector uses learning rate 5×10^{-4} , 40 epochs, and batch size 1024/1536; recovery uses learning rate 10^{-3} , batch size 1024, and 60 epochs. Early-stopping patience is 20–25 epochs for nominal policies, 10 for the detector, and 12 for recovery.

E. Hysteretic Control Arbitration

The controller enters recovery when $p_t \geq \tau_{\text{on}}$ and no cooldown is active. It remains in recovery for at least five steps. Thereafter, it releases control only after $p_t \leq 0.05$ for ten consecutive steps. A ten-step cooldown follows release, and a single intervention is capped at 250 recovery steps. Formally, the principal transitions are

$$z_t = N \rightarrow R \quad \text{if } p_t \geq \tau_{\text{on}} \text{ and } \text{cooldown} = 0, \quad (3)$$

$$z_t = R \rightarrow N \quad \text{if } n_R \geq 5 \text{ and } n_{\text{safe}} \geq 10. \quad (4)$$

The thresholds implement hysteresis: evidence required to enter recovery is intentionally much stronger than evidence required to remain nominal after a sustained safe period. ACT’s temporal state is reset whenever control returns from recovery. The broad 0.20 collection threshold prioritizes recovery-data coverage, whereas the higher deployment thresholds prioritize control selection and are frozen before confirmation evaluation.

IV. Experimental Protocol

A. Benchmarks and Data Separation

We use Meta-World 3.1.1 [19] with MuJoCo [20]. MT10 and MT50 provide 10 and 50 manipulation task families with known task identities. We use the goal-observable state representation and the official 500-step horizon. For each suite, task variants are reserved in distinct banks for demonstrations, detector training, recovery training, detector validation, and final evaluation. A digest-based audit verifies that the materialized task payloads and sampling-unit identities are disjoint across these stages.

TABLE I
Suite construction and final evaluation scale.

Quantity	MT10	MT50
Task families	10	50
Conditioned state dimension	49	89
Expert demonstrations	500	2,500
Failure-training rollouts	500	2,500
Recovery continuations	500	2,500
Detector-validation rollouts	200	1,000
Episodes/method/condition	500	2,500
Final task-macro units	10	50

Each task contributes 50 expert demonstrations, 50 failure-training rollouts, 50 retained recovery continuations, and 20 detector-validation rollouts. Final results use a separate confirmation bank with 50 episodes per task: 500 paired episodes per method for MT10 and 2,500 for MT50. Task variants, episode seeds, and injected noise arrays are shared across methods within a condition. No data from the confirmation bank are used to train a model or select a threshold.

Table I summarizes the scale of the two independently trained suites. The counts are deliberately symmetric per task; MT50 increases coverage by adding task families rather than by allocating fewer samples to each family. This design makes the MT10/MT50 comparison informative about breadth while avoiding a sample-budget confound at the task level.

B. Methods Compared

We compare four controllers. MT-MLP BC is a two-layer 256–256 policy trained on the same demonstrations as MT-ACT, the nominal action-chunking controller. Heuristic+Recovery uses the same learned recovery policy as REIM but triggers when simulator reward variation over a 20-step window falls below 0.01 after step 30. REIM replaces this reward-variation trigger with the learned monitor and hysteretic state machine while holding π_R fixed.

Table II summarizes the validation-selected detector operating points and the controller settings frozen before confirmation evaluation. The precision, recall, and expected calibration error (ECE) entries characterize near-term risk-event classification on the validation bank; they are not task-success metrics.

For an occupancy-matched control, validation selected heuristic tolerances of 0.06 (MT10) and 0.02 (MT50), using 20 episodes per task in both suites, to match canonical REIM occupancy at $\lambda = 0.40$; confirmation reused the same task variants, episode seeds, and recovery policy.

C. Clean and Perturbed Evaluation

Official-clean evaluation uses unmodified benchmark execution. Robustness is evaluated separately with task-universal independent Gaussian noise. For level λ ,

$$\epsilon_t^a \sim \mathcal{N}(0, (0.4\lambda)^2 I_4), \quad \epsilon_t^o \sim \mathcal{N}(0, (0.025\lambda)^2 I_{39}), \quad (5)$$

TABLE II

Validated detector operating points and frozen control settings.

Quantity	MT10	MT50
Trigger threshold	0.65	0.64
Task-macro precision	0.600	0.600
Task-macro recall	0.505	0.535
Task-macro ECE	0.138	0.141
Release threshold	0.05	0.05
Release patience (steps)	10	10
Minimum recovery (steps)	5	5
Cooldown (steps)	10	10
Recovery budget (steps)	250	250

with $\lambda \in \{0, 0.10, 0.20, 0.30, 0.40\}$. Actions are clipped to the environment range after perturbation. Object positions are never teleported in this multi-task protocol. At $\lambda = 0.40$, the action and observation standard deviations are 0.16 and 0.01. We report this sweep as a robustness extension to the benchmark.

The primary metric is task-macro success: success is averaged within each task and then equally across tasks. For 95% intervals, we resample episodes within each task 5,000 times and recompute the macro mean. These intervals quantify episode-level uncertainty conditional on fixed model parameters; Table IV separately reports variation across detector/recovery training seeds. Paired comparisons resample matched episode differences within task using the same task variant, seed, and noise realization. Recovery occupancy is the fraction of executed steps controlled by π_R .

For paired comparison against the heuristic gate, let s_i^R and s_i^H be binary outcomes for matched episode i . We report rescued episodes when $(s_i^R, s_i^H) = (1, 0)$ and harmed episodes when $(s_i^R, s_i^H) = (0, 1)$. Their normalized difference equals the paired success-rate change,

$$\Delta_{R-H} = \frac{1}{N} \sum_{i=1}^N (s_i^R - s_i^H) = \frac{N_{\text{rescued}} - N_{\text{harmed}}}{N}. \quad (6)$$

This decomposition retains the direction of every discordant episode and prevents an aggregate improvement from being mistaken for uniform benefit.

V. Results

A. Clean Performance Is Largely Preserved

At $\lambda = 0$, MT-ACT obtains 97.4% success on MT10 and 91.64% on MT50; REIM obtains 97.6% [96.2, 98.8] and 92.24% [91.4, 93.04]. The point differences are +0.2 and +0.6 points. Heuristic+Recovery is higher at 98.0% and 94.0%, whereas REIM’s advantage emerges under stronger perturbations. On clean episodes, REIM intervenes in 46.0%/28.64% of MT10/MT50 episodes but occupies only 18.70%/16.83% of executed steps.

B. A Common Perturbation Sweep Links MT10 and MT50

Figure 2 applies one perturbation definition and budget to fixed reference models from both suites. At $\lambda = 0.10$,

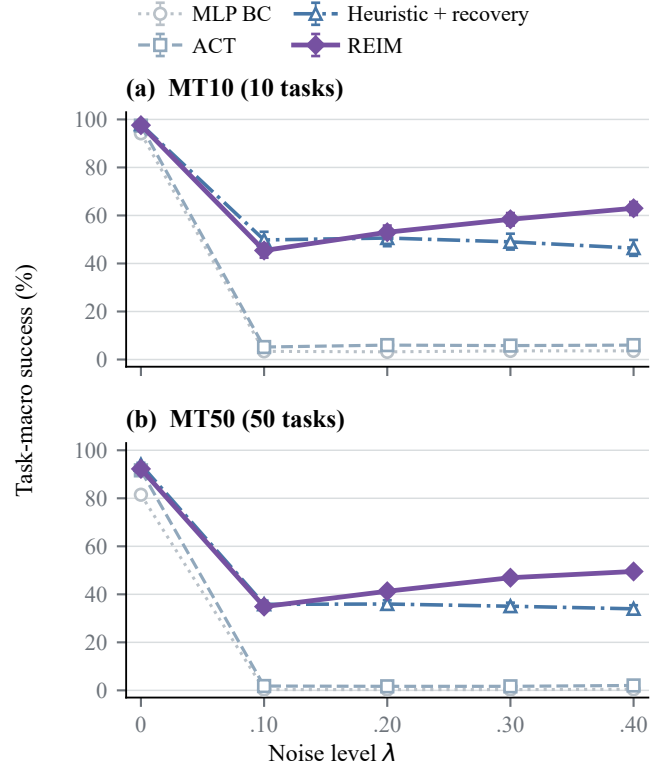


Fig. 2. Reference-model task-macro success under shared MT10/MT50 perturbations; 95% within-task bootstrap intervals, 50 episodes per task.

Heuristic+Recovery has the higher point estimate (49.8% versus 45.4% on MT10; 35.84% versus 34.88% on MT50). REIM has higher point estimates from $\lambda = 0.20$ onward and reaches advantages of 16.6 and 15.56 points at $\lambda = 0.40$, while ACT falls to 6.0% and 2.04%. REIM success rises with disturbance strength as control shifts toward recovery: occupancy increases from 37.35% to 69.21% on MT10 and from 45.09% to 67.40% on MT50 between $\lambda = 0.10$ and 0.40. The trend reflects the closed-loop switching response.

C. Strong Perturbations: Net Gains with Two-Sided Effects

Figure 3(a) gives the reference-model comparison at $\lambda = 0.40$. REIM reaches 63.0% [60.2, 65.6] on MT10 and 49.52% [48.16, 50.92] on MT50, versus 46.4% [43.2, 49.8] and 33.96% [32.52, 35.44] for Heuristic+Recovery. Because π_R is shared, the difference is attributable to the arbitration configuration, including when and how long recovery is used.

The paired decomposition in Fig. 3(b) shows 115 rescued and 32 harmed MT10 episodes, a net +16.60 points (paired 95% CI [12.80, 20.60]). MT50 has 610 rescued and 221 harmed episodes, a net +15.56 points [13.68, 17.44]. These gains accompany intervention rates of 93.4% and 98.68% and occupancies of 69.21% and 67.40%.

TABLE III
Canonical seed-42 occupancy-matched confirmation at $\lambda = 0.40$ (percentages).

Suite	Success R/H	Occ. R/H	Delta [95% CI] (pp)
MT10	63.0/63.2	69.21/67.46	-0.20 [-3.60, +3.00]
MT50	49.52/38.20	67.40/67.68	+11.32 [+9.44, +13.16]

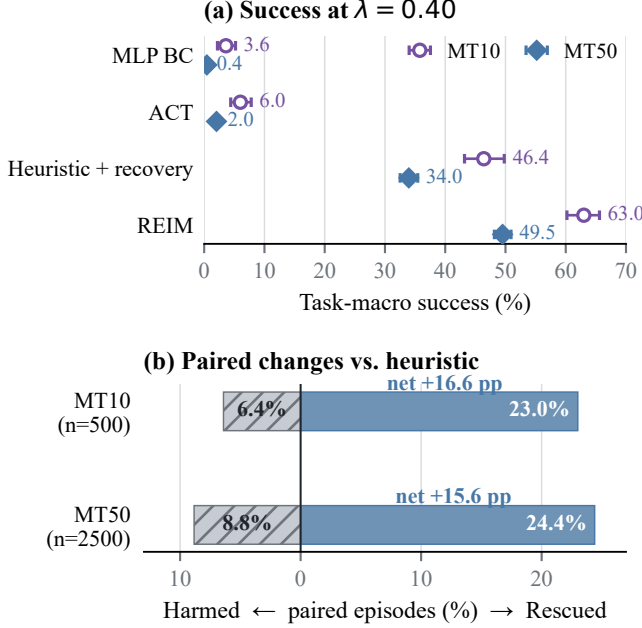


Fig. 3. Strong-perturbation results for the original fixed heuristic gate. (a) Success with 95% bootstrap intervals. (b) Paired rescued/harmed outcomes normalized by the number of paired episodes. The occupancy-matched confirmation is reported in the text.

The occupancy-matched confirmation separates usage from trigger selection. Table III summarizes the canonical seed-42 comparison; the learned trigger adds value at the broader task-set scale, but not on MT10 under this protocol.

D. Benefits Vary Across Task Families

Figure 4 shows that REIM improves 7/10 MT10 tasks and 32/50 MT50 tasks, ties 3 MT50 tasks, and is worse on the remaining 3 and 15 tasks. Both shared and MT50-specific families appear on either side of the diagonal, so the aggregate gain is distributed but not universal.

Door-open improves by 82 points in both suites, while MT50 stick-push, coffee-pull, and coffee-push decline by 50, 28, and 26 points. The negative cases are consistent with mistimed switching or recovery states poorly covered by successful continuations; the present experiment does not separate these mechanisms.

E. Component Training-Seed Stability

Table IV varies detector and recovery optimization seeds while fixing ACT, recovery-collection banks, thresholds, and 50-episode-per-task evaluation banks. REIM exceeds its same-seed original heuristic baseline for every MT10

● MT50-specific ○ Shared with MT10

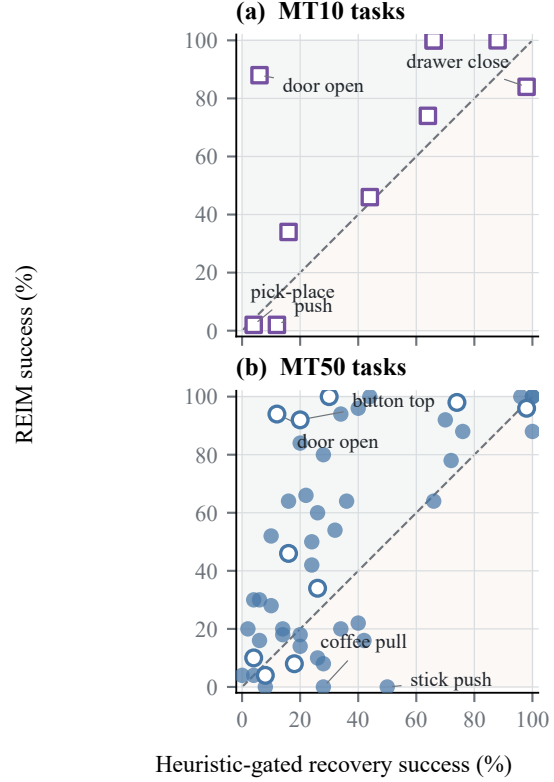


Fig. 4. Per-task success at $\lambda = 0.40$ (50 episodes per task). Above-diagonal points favor REIM; hollow MT50 markers denote shared tasks.

and MT50 seed at $\lambda = 0.40$; the mean advantages are 20.0 and 13.65 points. At $\lambda = 0.10$, both suite means remain below the heuristic, confirming that the gain is specific to stronger perturbations.

VI. Discussion and Limitations

The results show that learned arbitration is most useful after perturbations move nominal policies beyond their demonstrated state distribution. Unlike dataset-aggregation and noise-injection methods that alter nominal-policy training [1], [2], or RL-based recovery methods that optimize fallback behavior [9], [17], REIM keeps both policies fixed at test time and changes run-time control selection. The original operating point im-

TABLE IV
Component training-seed stability: task-macro success (mean $\pm$ SD, %).

Suite	λ	REIM	Heuristic
MT10 (3 seeds)	0	96.6 $\pm$ 0.4	96.7 $\pm$ 0.9
	0.10	44.1 $\pm$ 1.3	44.7 $\pm$ 3.2
	0.40	60.8 $\pm$ 4.9	40.8 $\pm$ 3.6
MT50 (4 seeds)	0	92.36 $\pm$ 0.35	93.46 $\pm$ 0.62
	0.10	34.39 $\pm$ 2.59	37.38 $\pm$ 1.31
	0.40	48.75 $\pm$ 1.05	35.10 $\pm$ 0.94

proves strong-noise success across both suites, while the occupancy-matched test shows a trigger-specific gain on MT50 but not MT10.

The occupancy-matched control removes the recovery-usage confound at $\lambda = 0.40$ and reveals a task-set-dependent trigger effect. Results remain limited to known-task Meta-World simulation, separately trained suites, Gaussian state-action noise, and successful expert continuations. Structured disturbances and physical-robot evaluation are next.

VII. Conclusion

REIM combines action-chunking imitation, causal near-term risk monitoring, supervised recovery, and hysteretic arbitration in one multi-task controller. It preserves clean performance and improves strong-noise success over nominal policies and the original heuristic gate. Occupancy matching shows a trigger gain on MT50 but not MT10; paired and per-task analyses expose rescued, harmed, and task-dependent outcomes.

References

- [1] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, ser. Proceedings of Machine Learning Research, vol. 15. PMLR, 2011, pp. 627–635. [Online]. Available: <https://proceedings.mlr.press/v15/ross11a.html>
- [2] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, “DART: Noise injection for robust imitation learning,” in Proceedings of the 1st Annual Conference on Robot Learning, ser. Proceedings of Machine Learning Research, vol. 78. PMLR, 2017, pp. 143–156. [Online]. Available: <https://proceedings.mlr.press/v78/laskey17a.html>
- [3] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in Proceedings of the 5th Conference on Robot Learning, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 1678–1690. [Online]. Available: <https://proceedings.mlr.press/v164/mandlekar22a.html>
- [4] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, “Implicit behavioral cloning,” in Proceedings of the 5th Conference on Robot Learning, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 158–168. [Online]. Available: <https://proceedings.mlr.press/v164/florence22a.html>
- [5] N. M. Shafullah, Z. Cui, A. Altanzaya, and L. Pinto, “Behavior transformers: Cloning k modes with one stone,” in Advances in Neural Information Processing Systems, vol. 35, 2022, pp. 22 955–22 968.
- [6] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. M. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in Proceedings of Robotics: Science and Systems, Daegu, Republic of Korea, Jul. 2023.
- [7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in Proceedings of Robotics: Science and Systems, Daegu, Republic of Korea, Jul. 2023.
- [8] J. Spencer, S. Choudhury, M. Barnes, M. Schmittle, M. Chiang, P. Ramadge, and S. Srinivasa, “Learning from interventions: Human-robot interaction as both explicit and implicit feedback,” in Proceedings of Robotics: Science and Systems, Corvallis, Oregon, USA, Jul. 2020.
- [9] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, “Recovery RL: Safe reinforcement learning with learned recovery zones,” IEEE Robotics and Automation Letters, vol. 6, no. 3, pp. 4915–4922, 2021.
- [10] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in 2019 International Conference on Robotics and Automation, 2019, pp. 6023–6029.
- [11] C. Wang, L. Fan, J. Sun, R. Zhang, L. Fei-Fei, D. Xu, Y. Zhu, and A. Anandkumar, “Mimicplay: Long-horizon imitation learning by watching human play,” in Proceedings of the 7th Conference on Robot Learning, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 201–221. [Online]. Available: <https://proceedings.mlr.press/v229/wang23a.html>
- [12] M. Shridhar, L. Manuelli, and D. Fox, “Perceiver-actor: A multi-task transformer for robotic manipulation,” in Proceedings of the 6th Conference on Robot Learning, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 785–799. [Online]. Available: <https://proceedings.mlr.press/v205/shridhar23a.html>
- [13] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan, “VIMA: Robot manipulation with multimodal prompts,” in Proceedings of the 40th International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 14 975–15 022. [Online]. Available: <https://proceedings.mlr.press/v202/jiang23b.html>
- [14] D. Kalashnikov, J. Varley, Y. Chebotar, B. Swanson, R. Jonschkowski, C. Finn, S. Levine, and K. Hausman, “MT-Opt: Continuous multi-task robotic reinforcement learning at scale,” 2021. [Online]. Available: <https://arxiv.org/abs/2104.08212>
- [15] C. Agia, R. Sinha, J. Yang, Z. Cao, R. Antonova, M. Pavone, and J. Bohg, “Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress,” in Proceedings of the 8th Conference on Robot Learning, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 689–723. [Online]. Available: <https://proceedings.mlr.press/v270/agia25a.html>
- [16] S. Vats, M. Likhachev, and O. Kroemer, “Efficient recovery learning using model predictive meta-reasoning,” in 2023 IEEE International Conference on Robotics and Automation, 2023, pp. 7258–7264.
- [17] S. Vats, D. K. Jha, M. Likhachev, O. Kroemer, and D. Romeres, “Recoverychaining: Learning local recovery policies for robust manipulation,” arXiv preprint arXiv:2410.13979, 2024. [Online]. Available: <https://arxiv.org/abs/2410.13979>
- [18] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [19] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning,” in Proceedings of the Conference on Robot Learning, ser. Proceedings of Machine Learning Research, vol. 100. PMLR, 2020, pp. 1094–1100. [Online]. Available: <https://proceedings.mlr.press/v100/yu20a.html>
- [20] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2012, pp. 5026–5033.